

Xcodeghost is an example of partial Trust attack (not fully), similar to what is described in the paper. Idea: the download of Xcode, in china, was too slow caused by the Great Firewall of China, so the developers started to download copies of the Apple Xcode development environment, and encountered altered versions that had been posted on domestic web sites. This opened the door for the malware to be inserted into high profile apps used on iOS devices.



Another example of how important it is to ensure the security of the software supply chain (compromising the compiler means compromising the entire supply chain)



TURING AWARD LECTURE

Dennis Ritchie, Ken Thompson; 1983: "For their development of generic operating systems theory and specifically for the implementation of the UNIX operating system"

It is called a "partial trust" attack (or supply chain attack) because:
- Trust in the Tool: The developer trusts the development tool (Xcode). If the tool is compromised, everything it produces is compromised.
- Bypassing the App Store: Since the apps were compiled with a near-official tool, they managed to bypass Apple's automated security checks and ended up on the official App Store, where they were downloaded by hundreds of millions of users.

Reflections on Trusting Trust

To what extent should one trust a statement that a program is free of Trojan horses? Perhaps it is more important to trust the people who wrote the software.

The takeaway of Ken Thompson article isn't "we can do nothing" (it is absolutely not of the kind there is no hope, trust is a problem, there is nothing that can be done). It is the bootstrap for thinking out of the box and for solving a problem that is obviously complicated but it will be needed to be solved otherwise we will rely on Trust and nothing more than Trust.

- Possible partial solutions:
- 1) Having a single compiler, very likely, isn't a good idea. Having multiple compilers can be a solution or a part of the solution. If we have a single compiler, we are trusting only that (can be compromised in the past and still now and it is difficult to know: so we are trusting not only the current version of the compiler but the full history of it). Having multiple compilers, permit you to have dependency on different chains of trust instead of being linked to a single one.
 - 2) Reproducible builds. Up to now, compilation isn't deterministic: it means that when you compile a program very likely you will obtain a binary (if you repeat this operation, the binary that you will obtain is different from the previous one: for many reasons like technical, etc.). This is really bad because being able to verify your Software supply-chain means that you must have a fingerprint of every single software (to achieve this compilation must be deterministic) and you must be able, every time, to rebuild the application and to check if somebody, for example, inserted a backdoor during compilation. To reduce this kind of problems, some people are suggesting to create builds that are reproducible. So, at this point, every single compilation/building process will generate a piece of software that can be generated again that is exactly identical to the previous one (but if the previous one was already compromised, you solved nothing)
 - 3) Monitoring. The backdoor inserted in the login program by a compromised compiler can do nothing if we aren't connected with the endpoint through the internet (the problem with backdoors is data exfiltration: the unauthorized transfer of sensitive information from within a corporate network to an external destination under the control of a malicious user). The typical solution is Monitoring because sooner or later, the login credentials will have to be sent to the attacker: I try to do my best to proper trust the compiler, but monitoring what happens in the system is Mitigation and a lot of security principle that we have seen are following this approach.

If you combine together Multiple Compilers with Reproducible builds, it can be a very good option to verify the behaviour of compilers (if two compilers use the same approach to perform a translation from source to binary and they produce different results, you can investigate) (if they give same result, they aren't compromised or they are all two compromised)



KEN THOMPSON

The lesson is that Trust is much more complex than expected because it has a lot of dependencies. For example, is not sufficient to check the source code of every program, that you used, to have trust in the software. Trust is the baseline of Cybersecurity (even if the professor dislikes this assumption). So, every time we are using a computer, we are not only trusting the hardware, the OS and whatever it is, we are also trusting special programs like compilers because every time we perform a compilation we are trusting a piece of software that can have really bad consequences

INTRODUCTION

I thank the ACM for this award. I can't help but feel that I am receiving this honor for timing and serendipity as much as technical merit. UNIX¹ swept into popularity with an industry-wide change from central mainframes to autonomous minis. I suspect that Daniel Bobrow [1] would be here instead of me if he could not afford a PDP-10 and had had to "settle" for a PDP-11. Moreover, the current state of UNIX is the result of the labors of a large number of people.

There is an old adage, "Dance with the one that brought you," which means that I should talk about UNIX. I have not worked on mainstream UNIX in many years, yet I continue to get undeserved credit for the work of others. Therefore, I am not going to talk about UNIX, but I want to thank everyone who has contributed.

That brings me to Dennis Ritchie. Our collaboration has been a thing of beauty. In the ten years that we have worked together, I can recall only one case of miscoordination of work. On that occasion, I discovered that we both had written the same 20-line assembly language program. I compared the sources and was astounded to find that they matched character-for-character. The result of our work together has been far greater than the work that we each contributed.

I am a programmer. On my 1040 form, that is what I put down as my occupation. As a programmer, I write

programs. I would like to present to you the cutest program I ever wrote. I will do this in three stages and try to bring it together at the end.

STAGE I

In college, before video games, we would amuse ourselves by posing programming exercises. One of the favorites was to write the shortest self-reproducing program. Since this is an exercise divorced from reality, the usual vehicle was FORTRAN. Actually, FORTRAN was the language of choice for the same reason that three-legged races are popular.

More precisely stated, the problem is to write a source program that, when compiled and executed, will produce as output an exact copy of its source. If you have never done this, I urge you to try it on your own. The discovery of how to do it is a revelation that far surpasses any benefit obtained by being told how to do it. The part about "shortest" was just an incentive to demonstrate skill and determine a winner.

Figure 1 shows a self-reproducing program in the C³ programming language. (The purist will note that the program is not precisely a self-reproducing program, but will produce a self-reproducing program.) This entry is much too large to win a prize, but it demonstrates the technique and has two important properties that I need to complete my story: 1) This program can be easily written by another program. 2) This program can contain an arbitrary amount of excess baggage that will be reproduced along with the main algorithm. In the example, even the comment is reproduced.

Can Computer program learn without Data? (Attention: we are not talking about AI)
The compiler is a special kind of program because it is a program that must learn. If it isn't possible for a compiler to learn, it isn't possible to create a new version of the compiler with better features in respect of the previous version of the compiler

So, yes, it is possible to create a program that is able to learn and for implementing the learning (add new features) i use a version of the compiler to compile the next version. We will see that this learning mechanism (compile the new version of the compiler with the previous one) can be abused by an attacker.

¹ UNIX is a trademark of AT&T Bell Laboratories.

The bootstrap problem (being able to bootstrap a compiler when you don't have a compiler): if i need a compiler for compiling the next version, where does it come from the first version of the compiler? How it is possible to compile the first version of the compiler when i create the new architecture or the first PC? The answer to this problem is, typically used, cross-compilation that is "using an architecture that is different from your target one for compiling a binary for another architecture" or, another solution, writing the first mock-up of a compiler using a low-level language (example: assembly). So, the first version of the compiler is really really simple that permits you to exit from the bootstrapping problem and, after bootstrapping, evolve it with high-language, if possible (example: GCC, c compiler, that exit the bootstrapping problem and then evolved using C language)

Thompson dimostra che una volta che una "conoscenza" (o un comportamento) è stata inserita nel binario del compilatore, il codice sorgente può essere cambiato o ripulito, ma il compilatore continuerà a portarsi dietro quella nozione. Questo è il "ponte" fondamentale per lo Stadio III: se puoi insegnargli un simbolo innocuo come `lv`, puoi insegnargli segretamente a inserire una `backdoor` senza che questa appaia mai più nel codice sorgente.

```

char s[ ] = {
    '\t',
    '0',
    '\n',
    '}',
    ',',
    '\n',
    '\n',
    '/',
    ',',
    '\n',
    (213 lines deleted)
    0
};

/*
 * The string s is a
 * representation of the body
 * of this program from '0'
 * to the end.
 */

main( )
{
    int i;

    printf("char\ts[ ] = {\n");
    for(i=0; s[i]; i++)
        printf("\t%d, \n", s[i]);
    printf("%s", s);
}

```

Here are some simple transliterations to allow a non-C programmer to read this code.

=	assignment
==	equal to .EQ.
!=	not equal to .NE.
++	increment
'x'	single character constant
"xxx"	multiple character string
%d	format to convert to decimal
%s	format to convert to string
\t	tab character
\n	newline character

FIGURE 1.

STAGE II

The C compiler is written in C. What I am about to describe is one of many “chicken and egg” problems that arise when compilers are written in their own language. In this case, I will use a specific example from the C compiler.

C allows a string construct to specify an initialized character array. The individual characters in the string can be escaped to represent unprintable characters. For example,

```
“Hello world\n”
```

represents a string with the character “\n,” representing the new line character.

Figure 2.1 is an idealization of the code in the C compiler that interprets the character escape sequence. This is an amazing piece of code. It “knows” in a completely portable way what character code is compiled for a new line in any character set. The act of knowing

then allows it to recompile itself, thus perpetuating the knowledge.

Suppose we wish to alter the C compiler to include the sequence “\v” to represent the vertical tab character. The extension to Figure 2.1 is obvious and is presented in Figure 2.2. We then recompile the C compiler, but we get a diagnostic. Obviously, since the binary version of the compiler does not know about “\v,” the source is not legal C. We must “train” the compiler. After it “knows” what “\v” means, then our new change will become legal C. We look up on an ASCII chart that a vertical tab is decimal 11. We alter our source to look like Figure 2.3. Now the old compiler accepts the new source. We install the resulting binary as the new official C compiler and now we can write the portable version the way we had it in Figure 2.2.

This is a deep concept. It is as close to a “learning” program as I have seen. You simply tell it once, then you can use this self-referencing definition.

STAGE III

Again, in the C compiler, Figure 3.1 represents the high level control of the C compiler where the routine “com-

```

...
c = next( );
if(c != '\\')
    return(c);
c = next( );
if(c == '\\')
    return('\\\\');
if(c == 'n')
    return('\\n');
...

```

FIGURE 2.2.

```

...
c = next( );
if(c != '\\')
    return(c);
c = next( );
if(c == '\\')
    return('\\');
if(c == 'n')
    return('\n');
if(c == 'v')
    return('\v');
...

```

FIGURE 2.1.

```

c = next( );
if(c != '\\')
    return(c);
c = next( );
if(c == '\\')
    return('\\');
if(c == 'n')
    return(' n ');
if(c == 'v')
    return('11');

```

FIGURE 2.3.

In C, una "stringa" è tecnicamente un array di caratteri.

Idea of Stage 3.1: there is a malicious attacker that is willing to insert a backdoor (in the paper, Trojan horse), a piece of code, to intercept username and password of one of the users that are inside the system and, for being able to do that, one of the different possibilities (possibility that isn't included in the paper) is to insert a sniffer/backdoor that is inside the OS that is able to get the credentials that are typed from the user on the keyboard during the login (in every OS, there is a program that is in charge of getting the user and password, that is called `login` in Unix/Linux).

How to do it?

762 *Communications of the ACM*

- 1) Insert the backdoor in the source code of the login program (in Unix/Linux) and spread the binary of the Login

- Drawback: leaving a lot of traces (backdoor inside the source code (that is not hidden)) that can be found

2) Insert the backdoor in the compiler, instead of Login, so it will add the backdoor in the program compiled (add extra binary code to the source code of login). Very likely they will not analyse the compiler because they will analyse only the login

- Drawback: leaving a lot of traces in the source code of the compiler that is not hidden

Morale (cosa vuole dirvi Thompson?)

Verso l'alto (alle aziende/esperti): Smettete di pensare che basti guardare il codice sorgente per essere sicuri. La sicurezza è una questione di catena di fiducia umana, non solo di algoritmi.

Verso il basso (ai giovani hacker): Smettete di pensare che "bucare" sistemi sia un gioco o una prova di intelligenza. È un crimine che danneggia la società e la cultura informatica.

L' inutilità dell'analisi (Scrutiny): Thompson afferma che puoi leggere il codice sorgente (il testo) di un programma per anni, ma non sarai mai sicuro che sia "pulito". Perché? Perché il programma che trasforma quel testo in qualcosa di eseguibile (il compilatore) potrebbe essere corrotto.

Turing Award Lecture

Per entrare in un sistema UNIX, devi conoscere la password criptata di un utente. Con questo trucco, l'attaccante può loggarsi come qualsiasi utente (incluso l'amministratore e "root") usando una password segreta che solo lui conosce.

Se il primo Trojan serviva ad attaccare il sistema (il comando login), questo secondo Trojan serve a proteggere il segreto attaccando il compilatore stesso

2. La Fase di "Apprendimento" (Bootstrapping) Come hai visto nello Stadio II, il compilatore deve essere "addestrato".

Passaggio A: Prendi il codice sorgente del compilatore C e scrivilo dentro, in chiaro, le istruzioni per i due Trojan.

Passaggio B: Compila questo codice usando un compilatore C normale (pulito).

Risultato: Otteni un binario infetto (un file eseguibile) che ora "sa" come inserire i bug.

3. L'Invisibilità Totale Una volta installato questo binario infetto come "compilatore ufficiale", accade qualcosa di incredibile:

Pulizia: Thompson può ora cancellare tutte le linee di codice malevole dai file sorgente del compilatore. Il codice sorgente torna a sembrare onesto al 100%.

Persistenza: Quando il programmatore userà il binario infetto per compilare il codice sorgente "pulito", il binario riconoscerà che sta compilando un compilatore e re-inietterà i Trojan nel nuovo binario prodotto.

pile" is called to compile the next line of source. Figure 3.2 shows a simple modification to the compiler that will deliberately miscompile source whenever a particular pattern is matched. If this were not deliberate, it would be called a compiler "bug." Since it is deliberate, it should be called a "Trojan horse."

The actual bug¹ planted in the compiler would match code in the UNIX "login" command. The replacement code would miscompile the login command so that it would accept either the intended encrypted password or a particular known password. Thus if this code were installed in binary and the binary were used to compile the login command, I could log into that system as any user.

Such blatant code would not go undetected for long. Even the most casual perusal of the source of the C compiler would raise suspicions.

The final step is represented in Figure 3.3. This simply adds a second Trojan horse to the one that already exists. The second pattern is aimed at the C compiler. The replacement code is a Stage I self-reproducing program that inserts both Trojan horses into the compiler. This requires a learning phase as in the Stage II example. First we compile the modified source with the normal C compiler to produce a bugged binary. We install this binary as the official C. We can now remove the bugs from the source of the compiler and the new binary will reinsert the bugs whenever it is compiled. Of course, the login command will remain bugged with no trace in source anywhere.

```
compile(s)
char *s;
{
    ...
}
```

FIGURE 3.1.

```
compile(s)
char *s;
{
    if(match(s, "pattern")) {
        compile("bug");
        return;
    }
    ...
}
```

FIGURE 3.2.

```
compile(s)
char *s;
{
    if(match(s, "pattern1")) {
        compile("bug1");
        return;
    }
    if(match(s, "pattern 2")) {
        compile("bug 2");
        return;
    }
    ...
}
```

FIGURE 3.3.

Il codice auto-replicante serve per far sì che il compilatore figlio del padre malevolo venga infettato e infetti i successivi compilatori da compilare

MORAL

La sicurezza assoluta è un'illusione. Poiché nessuno può scriversi da solo l'intero sistema operativo, il compilatore, i driver e progettarsi i chip, siamo costretti a fidarci delle persone. La fiducia non è nel software, ma in chi lo produce.

The moral is obvious. You can't trust code that you did not totally create yourself. (Especially code from companies that employ people like me.) No amount of source-level verification or scrutiny will protect you from using untrusted code. In demonstrating the possibility of this kind of attack, I picked on the C compiler. I could have picked on any program-handling program such as an assembler, a loader, or even hardware microcode. As the level of program gets lower, these bugs will be harder and harder to detect. A well-installed microcode bug will be almost impossible to detect.

After trying to convince you that I cannot be trusted, I wish to moralize. I would like to criticize the press in its handling of the "hackers," the 414 gang, the Dalton gang, etc. The acts performed by these kids are vandalism at best and probably trespass and theft at worst. It is only the inadequacy of the criminal code that saves the hackers from very serious prosecution. The companies that are vulnerable to this activity, (and most large companies are very vulnerable) are pressing hard to update the criminal code. Unauthorized access to computer systems is already a serious crime in a few states and is currently being addressed in many more state legislatures as well as Congress.

There is an explosive situation brewing. On the one hand, the press, television, and movies make heroes of vandals by calling them whiz kids. On the other hand, the acts performed by these kids will soon be punishable by years in prison.

I have watched kids testifying before Congress. It is clear that they are completely unaware of the seriousness of their acts. There is obviously a cultural gap. The act of breaking into a computer system has to have the same social stigma as breaking into a neighbor's house. It should not matter that the neighbor's door is unlocked. The press must learn that misguided use of a computer is no more amazing than drunk driving of an automobile.

Acknowledgment. I first read of the possibility of such a Trojan horse in an Air Force critique [4] of the security of an early implementation of Multics. I cannot find a more specific reference to this document. I would appreciate it if anyone who can supply this reference would let me know.

REFERENCES

1. Bobrow, D.G., Burchfiel, J.D., Murphy, D.L., and Tomlinson, R.S. TENEX, a paged time-sharing system for the PDP-10. *Commun. ACM* 15, 3 (Mar. 1972), 135-143.
2. Kernighan, B.W., and Ritchie, D.M. *The C Programming Language*. Prentice-Hall, Englewood Cliffs, N.J., 1978.
3. Ritchie, D.M., and Thompson, K. The UNIX time-sharing system. *Commun. ACM* 17, (July 1974), 365-375.
4. Unknown Air Force Document.

Author's Present Address: Ken Thompson, AT&T Bell Laboratories, Room 2C-519, 600 Mountain Ave., Murray Hill, NJ 07974.

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires a fee and/or specific permission.

Se non è il compilatore, potrebbe essere l'Assembler. Se non è l'Assembler, potrebbe essere il microcodice (il software scritto direttamente nei chip hardware). Più scendi in basso nei livelli del computer, più il "bug" diventa invisibile.

The binary version of the compiler is very hard to be verified by a human being, so the only thing that we can do is to trust the compiler

Idea of Stage 3.2: instead of inserting only one backdoor in the compiler, we insert two backdoors: the new backdoor is "every time the compiler recognises that it is compiling a compiler, insert this backdoor and the first one that is about the login program". So, the malicious compiler will insert the two backdoors in every next versions of the compiler. Now, I compile the new version of the compiler and remove the backdoors in the source code of the compiler because they are inserted in the binary version of the compiler and I got two points in terms of compromise of the system: 1) Persistence: compromise is inherited in the new version every time I compile the source code of the new version of the compiler; 2) Targeting: the malicious compiler insert a backdoor only in program that are important (adding to the source code the code for the backdoor)